

JWT (JSON Web Token)

JWT (JSON Web Token) is a compact, secure, and self-contained token format used for **authentication, authorization, and secure data exchange** between client and server.

It is one of the most common authentication mechanisms in modern web applications, APIs, microservices, and mobile apps.

1. Core Idea

Instead of storing user session data on the server:

None

Server stores session in DB/cache

JWT stores verified claims inside a signed token:

None

Client stores token

Client sends token with requests

Server verifies token

This enables **stateless authentication**.

2. JWT Structure

A JWT has 3 parts separated by dots:

None

HEADER.PAYLOAD.SIGNATURE

Example:

None

eyJhbGciOiI...eyJ1c2VySWQi...abc123xyz

3. JWT Components

Header

Contains metadata:

None

```
{  
  
  "alg": "HS256",  
  
  "typ": "JWT"  
}
```

Meaning:

- **alg** = signing algorithm
- **typ** = token type

Payload

Contains claims / data:

None

```
{  
  
  "userId": 101,  
  
  "role": "admin",  
  
  "exp": 1715000000  
}
```

Common claims:

None

```
sub = subject  
  
iat = issued at  
  
exp = expiry  
  
iss = issuer  
  
aud = audience
```

⚠ Payload is encoded, **not encrypted**.

Signature

Used to verify integrity:

None

```
HMACSHA256(  
  
    base64(header) + "." + base64(payload),  
  
    secret_key  
  
)
```

If payload changes, signature becomes invalid.

4. How JWT Authentication Works

Login Flow

None

1. User enters email/password
2. Server validates credentials
3. Server generates JWT
4. Client stores token

Request Flow

None

GET /profile

Authorization: Bearer <token>

Server:

None

1. Verify signature

2. Check expiry

3. Extract userId / role

4. Process request

5. Diagram

None

User Login



Server validates credentials



Generate JWT



Client stores token



Future Requests:

Authorization: Bearer JWT



Server verifies token

6. Why JWT is Popular

✓ Stateless Authentication

Server does not need session storage.

None

No session DB lookup required

✓ Scalable

Great for distributed systems and microservices.

✓ Cross Platform

Works with:

None

Web

Mobile

SPA

Backend APIs

✓ Fast Authorization

Role/permissions can be inside token.

7. Common Use Cases

None

User login authentication

REST APIs

Microservices auth

SSO systems

Mobile apps

WebSocket authentication

8. JWT in System Design

For APIs:

None

Client → API Gateway → Services

Each service can verify JWT independently.

No central session store required.

9. Access Token + Refresh Token Model

Modern systems use:

Access Token

None

Short expiry (15 min)

Used for API calls

Refresh Token

None

Long expiry (days/weeks)

Used to issue new access token

This improves security.

10. JWT vs Session Authentication

Feature	Session	JWT
Server Stores State	Yes	No

Scalable	Medium	High
DB/Redis Needed	Yes	No
Easy Logout	Yes	Harder
Microservices Friendly	Medium	Excellent

11. Important Security Concerns

Do Not Store Sensitive Data

JWT payload is readable after decoding.

Bad:

None

password

credit card

OTP

Token Theft Risk

If attacker steals token, they can impersonate user.

Use:

None

HTTPS

Secure cookies

HttpOnly cookies

Short expiry

Logout is Harder

Since token is stateless:

None

Server cannot instantly revoke all issued tokens easily

Solutions:

- Token blacklist
- Short token expiry
- Refresh token rotation

Large Payloads

Too much data in token increases request size.

12. JWT Storage Options

Browser

None

`HttpOnly Secure Cookie` ← safest

`localStorage` ← common but XSS risk

`sessionStorage`

13. JWT for WebSocket

During connection:

None

`wss://chat.com?token=jwt`

or send token in auth handshake.

Server verifies before allowing connection.

14. Interview Insight

If interviewer asks:

Why JWT for microservices?

Answer:

None

JWT is stateless, self-contained, and easily verifiable,
so multiple services can authenticate requests
without centralized session storage.

15. When NOT to Use JWT

Avoid JWT when:

None

Need instant logout everywhere
Need server-side session control
Simple monolith app
Very sensitive short-lived sessions

Then session auth may be better.

16. One-Line Summary

JWT is a signed token containing user claims that enables stateless, scalable authentication and authorization across web apps, APIs, and distributed systems.